

PCA y Sistemas de Recomendación

Análisis de Componentes Principales aplicado al dataset Breast Cancer Wisconsin

Este tutorial explica paso a paso cómo utilizar PCA (Análisis de Componentes Principales) como técnica de preprocesamiento para clasificación, cómo interpretar la varianza explicada y los loadings, y cómo construir un sistema de recomendación basado en similitud usando el mismo dataset.

El tutorial está diseñado para que cualquier persona pueda reproducirlo en su propia computadora, siguiendo las celdas en orden.

Parte 1 — Exploración del Dataset

¿Qué vamos a hacer?

Cargar el dataset **Breast Cancer Wisconsin** e inspeccionarlo para entender con qué datos estamos trabajando antes de aplicar cualquier algoritmo.

¿Por qué?

Antes de modelar, siempre hay que responder: ¿cuántos datos hay? ¿están limpios? ¿las clases están balanceadas? También entrenaremos un modelo **baseline** (sin PCA) para tener un punto de comparación justo.

El dataset

- **569 muestras** de tumores (pacientes)
- **30 características** numéricas por tumor: radio, textura, perímetro, área, suavidad, etc.
- **2 clases**: 0 = Maligno y 1 = Benigno

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

np.random.seed(42)

# --- Cargar dataset ---
cancer_data = load_breast_cancer()
X = pd.DataFrame(cancer_data.data, columns=cancer_data.feature_names)
y = cancer_data.target # 0 = Maligno, 1 = Benigno

# --- Información general ---
print(f"Tipo de datos : {X.dtypes.value_counts().to_dict()}")
print(f"Total de valores nulos : {X.isnull().sum().sum()}")

classes = np.bincount(y)
print(f"Clases - Maligno (0): {classes[0]} | Benigno (1): {classes[1]}")

# --- Baseline sin PCA ---
clf_base = RandomForestClassifier(random_state=42)
scores = cross_val_score(clf_base, X, y, cv=5)
print(f"Accuracy baseline (sin PCA): {scores.mean():.4f} +/- {scores.std():.4f}")
print(f"Este número es nuestro punto de referencia. Después de aplicar PCA")
print(f"veremos si podemos mantener un rendimiento similar con menos dimensiones.")

X.head()
```

Tamaño (filas, columnas): (569, 30)
Tipo de datos : (dtype='float64'): 30
Total de valores nulos : 0
Clases - Maligno (0): 212 | Benigno (1): 357

Accuracy baseline (sin PCA): 0.9561 +/- 0.0228
Este número es nuestro punto de referencia. Después de aplicar PCA
veremos si podemos mantener un rendimiento similar con menos dimensiones.

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.60	2019.0	0.1622	0.6556	0.7119	0.2654	0.4601	0.11890
1	20.57	17.77	132.80	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902
2	19.69	21.25	130.00	1203.0	0.10960	0.10990	0.1974	0.12790	0.2069	0.05999	...	23.07	25.03	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.00	98.87	567.7	0.2098	0.8963	0.6899	0.2575	0.6638	0.17300
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	25.54	16.87	152.30	1575.0	0.1374	0.3050	0.4000	0.1625	0.2364	0.07678

5 rows x 30 columns

¿Qué nos dicen estos resultados?

- **Sin valores nulos**: el dataset está limpio, no necesitamos imputación.
- **212 malignos vs 357 benignos**: hay un ligero desbalance (63% / 37%), lo tendremos en cuenta.
- **Baseline ~96%**: con las 30 variables originales y Random Forest se clasifica muy bien. Este es el estándar que queremos mantener al reducir dimensiones con PCA.

Visualizaciones exploratorias — Dispersión y densidad

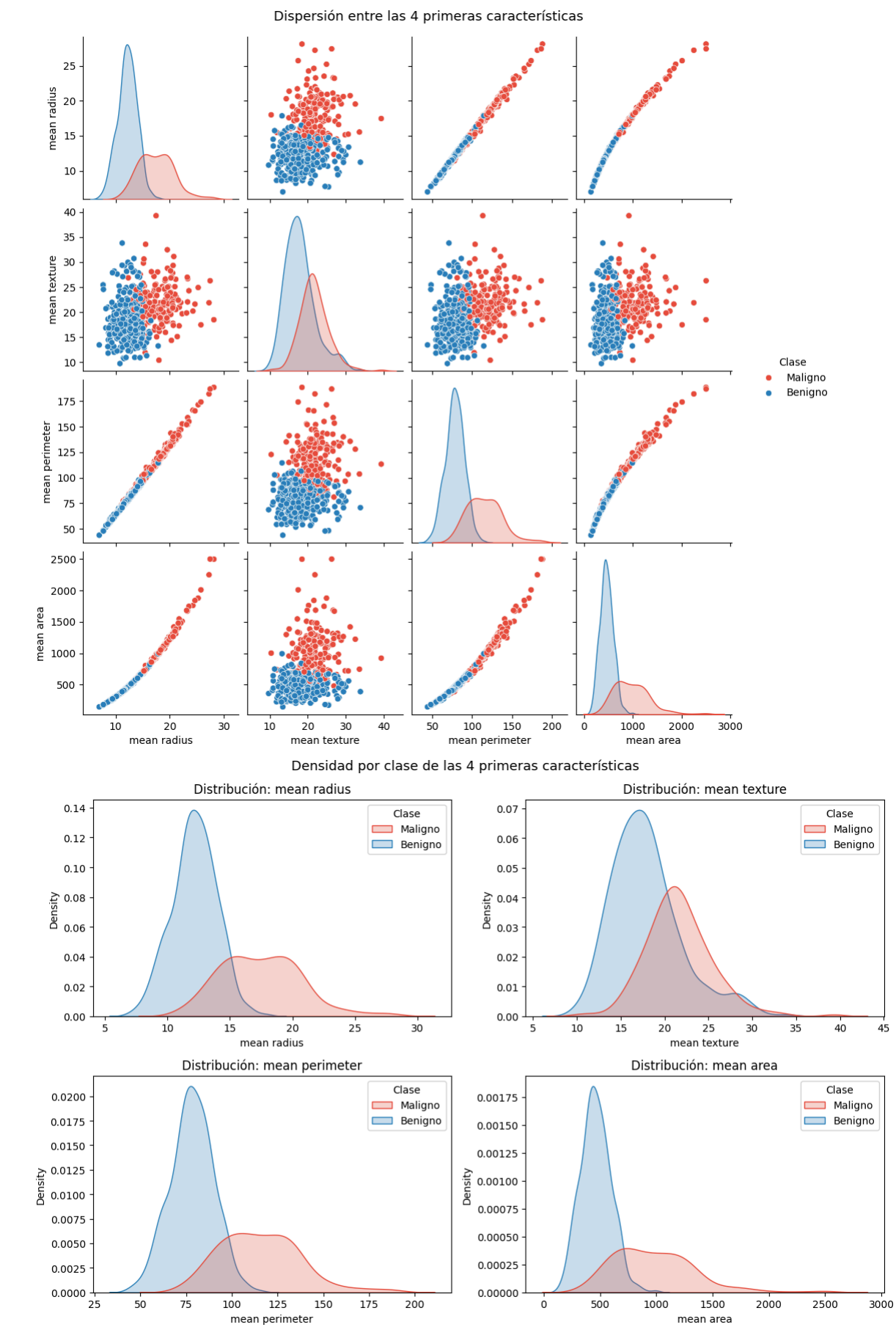
Antes de reducir dimensiones, visualizamos las **4 primeras características** del dataset para ver si ya existe alguna separación natural entre clases.

- **Gráfica de dispersión (pairplot)**: muestra la relación entre pares de variables, coloreando cada clase.
- **Gráfica de densidad (KDE)**: muestra la distribución de cada variable por clase. Si las curvas se solapan poco, esa variable es discriminativa.

```
# Seleccionamos las primeras 4 variables para visualizar
cols_to_plot = list(cancer_data.feature_names[:4])
df_plot = X[cols_to_plot].copy()
df_plot['Class'] = pd.Series(y).map({0: 'Maligno', 1: 'Benigno'})

# --- Pairplot ---
sns.pairplot(df_plot, hue='Class', palette='Maligno: #e7433c', 'Benigno: #298b99')
plt.suptitle('Dispersión entre las 4 primeras características', y=1.02, fontsize=13)
plt.show()

# --- Densidad (KDE) ---
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for i, feature in enumerate(cols_to_plot):
    sns.kdeplot(data=df_plot, x=feature, hue='Class',
                palette='Maligno: #e7433c', 'Benigno: #298b99',
                fill=True, ax=axes.flatten()[i])
axes.flatten()[3].set_title('Distribución: (feature)')
plt.suptitle('Densidad por clase de las 4 primeras características', fontsize=13)
plt.tight_layout()
plt.show()
```



Interpretación de las visualizaciones exploratorias

En las gráficas de densidad se puede ver que variables como `mean_radius` y `mean_area` muestran una **separación clara entre clases**: los tumores malignos tienden a tener valores más altos. Esto nos dice que estas variables son discriminativas y útiles para clasificar.

Sin embargo, con 30 variables es imposible visualizar todas las relaciones al mismo tiempo. Ahí es donde entra PCA.

Parte 2 — Preprocesamiento: División Train/Test y Estandarización

¿Qué vamos a hacer?

Dividir el dataset en conjuntos de entrenamiento y prueba, y estandarizar las variables.

¿Por qué dividir en Train y Test?

Si entrenamos y evaluamos con los mismos datos, el modelo "memoriza" en lugar de aprender. Para medir el rendimiento real necesitamos datos que el modelo **nunca haya visto** durante el entrenamiento.

¿Por qué estandarizar? (Esto es OBLIGATORIO antes de PCA)

PCA calcula direcciones de máxima varianza. Si `area_mean` tiene valores en miles (ej: 1200) y `smoothness_mean` tiene valores decimales (ej: 0.10), PCA pensará que `area` es más importante **solo por su escala numérica**, no por su contenido real.

`StandardScaler` transforma cada variable para que tenga **media = 0 y desviación estándar = 1**.

Regla importante: el scaler se ajusta (`fit`) solo con datos de entrenamiento y luego se aplica (`transform`) a ambos conjuntos. Esto evita que información del test "se filtre" al entrenamiento.

```
# --- División 70% train / 30% test ---
# stratify mantiene la proporción de clases en ambos conjuntos
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

print(f"--- TAMAÑOS ---")
print(f"Total : {X.shape[0]} muestras, {X.shape[1]} variables")
print(f"Train : {X_train.shape[0]} muestras (70%)")
print(f"Test : {X_test.shape[0]} muestras (30%)")

# --- Estandarización ---
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # aprende media/std de train y transforma
X_test_scaled = scaler.transform(X_test) # aplica lo aprendido en train, NO hace fit

print(f"--- VERIFICACIÓN (sobre Train) ---")
print(f"Media (debe ser = 0): {X_train_scaled.mean():.8f}")
print(f"Std (debe ser = 1): {X_train_scaled.std():.8f}")
print(f"✅ Datos listos para PCA")

# TAMAÑOS
Total : 569 muestras, 30 variables
Train : 398 muestras (70%)
Test : 171 muestras (30%)

# --- VERIFICACIÓN (sobre Train) ---
Media (debe ser = 0): -0.00000000
Std (debe ser = 1): 1.00000000

✅ Datos listos para PCA
```

Parte 3 — Visualización del Dataset: Heatmap de Datos Crudos

¿Qué vamos a hacer?

Generar un heatmap (mapa de calor) de todas las variables estandarizadas para los 569 pacientes.

¿Por qué?

Esta visualización nos permite ver de un solo vistazo el "estado crudo" de los datos:

- Colores rojos intensos → valores muy por encima del promedio
- Colores azules intensos → valores muy por debajo del promedio

Al ordenar las muestras por clase (Maligno/Benigno), veremos si los patrones de color difieren entre grupos, lo cual confirmaría que las variables contienen información discriminativa.



Interpretación del Heatmap

Se puede observar que la columna `TARGET` (extremo derecho) actúa como separador visual. Las muestras malignas (valor -3, azul intenso) y benignas (valor +3, rojo intenso) muestran **patrones de color distintos** en varias columnas, especialmente en las relacionadas con tamaño (`area`, `radius`, `perimeter`).

Esto confirma visualmente lo que vimos en las gráficas de densidad: las características del dataset sí contienen información útil para distinguir entre tipos de tumor. PCA buscará capturar esa información en menos dimensiones.

Parte 4 — Aplicación de PCA y Comparación de Correlaciones

¿Qué vamos a hacer?

Aplicar PCA reduciendo a 10 componentes y comparar la **matriz de correlación** antes y después de aplicarlo.

¿Por qué?

Una de las propiedades matemáticas más importantes de PCA es que produce componentes **ortogonales entre sí**, es decir, **no correlacionados**. En los datos originales hay mucha redundancia (por ejemplo, `radius` y `area` están altamente correlacionados).

Al comparar los heatmaps de correlación, veremos esto de forma muy clara:

- **Datos originales:** colores fuertes → alta correlación entre variables
- **Componentes PCA:** solo la diagonal tiene color → correlación = 0 entre componentes

```
# --- Aplicar PCA a 10 componentes ---
n_comp = 10
pca = PCA(n_components=n_comp, random_state=42)
X_pca = pca.fit_transform(X_train_scaled)

# Columnas = ['PC1(+1)' for i in range(n_comp)]
df_pca = pd.DataFrame(X_pca, columns=[f'PC{i+1}' for i in range(n_comp)])

varianza_total = np.sum(pca.explained_variance_ratio_) * 100
print(f'Con {n_comp} componentes se retiene el {varianza_total:.2f}% de la información original.')

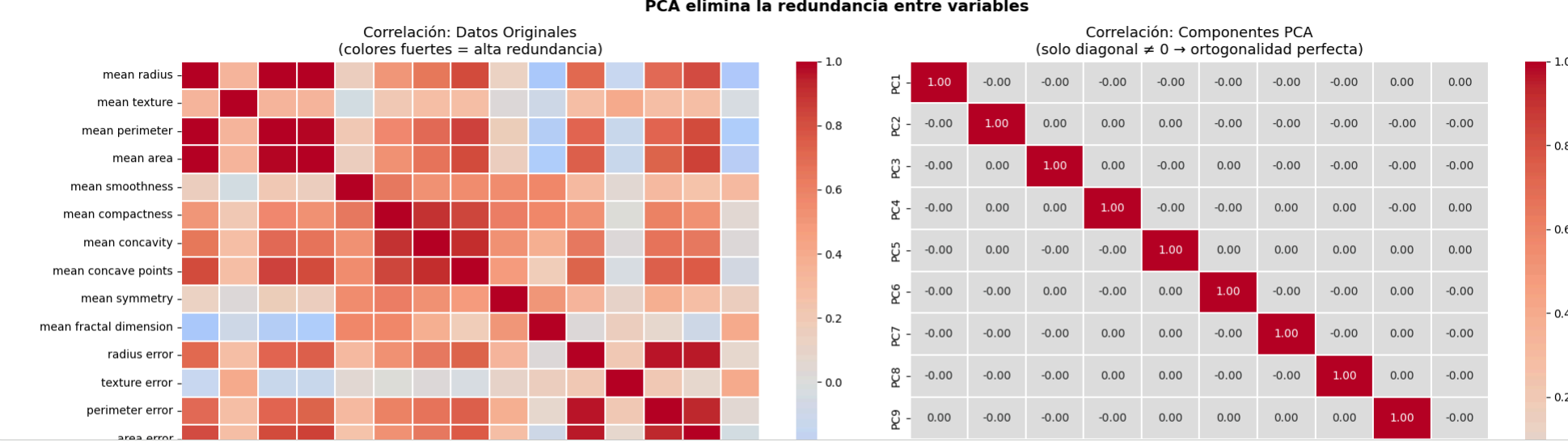
# --- Comparación de heatmaps de correlación ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 8))

# Heatmap 1: Datos originales (primeras 15 variables)
corr_orig = pd.DataFrame(X_train_scaled, columns=X_train_scaled.columns[:15], index=X_train_scaled.columns[:15])
sns.heatmap(corr_orig, annot=False, cmap='coolwarm', ax=ax1, center=0, linewidth=0.3)
ax1.set_title('Correlación: Datos Originales (colores fuertes = alta redundancia)', fontsize=13)

# Heatmap 2: Componentes PCA
corr_pca = df_pca.corr()
sns.heatmap(corr_pca, annot=True, fmt='.2f', cmap='coolwarm', ax=ax2, center=0, linewidth=0.3)
ax2.set_title('Correlación: Componentes PCA (solo diagonal = 0 = ortogonalidad perfecta)', fontsize=13)

plt.suptitle('PCA elimina la redundancia entre variables', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

Con 10 componentes se retiene el 95.62% de la información original.



Interpretación de la comparación de correlaciones

Datos originales (Izquierda): el heatmap muestra muchas celdas con colores rojos o azules intensos. Por ejemplo, mean_radius, mean_perimeter y mean_area están altamente correlacionados (casi 1.0) porque las tres miden variantes del tamaño del tumor. Esta redundancia significa que estamos usando 3 columnas para guardar esencialmente la misma información.

Componentes PCA (derecha): el heatmap muestra que la correlación entre cualquier par de componentes distintos es prácticamente cero. Esto es la **ortogonalidad** de PCA en acción. Cada componente captura una dirección de variación completamente independiente, sin redundancia.

Conclusión: PCA comprime la información eliminando duplicidad. Con 10 componentes guardamos más del 95% de la información usando solo un tercio de las variables originales.

Parte 5 — Análisis de Varianza Explicada: Scree Plot y Umbrales

¿Qué vamos a hacer?

Analizar cuánta información captura cada componente principal, tanto de forma individual como acumulada.

¿Por qué?

Esta es la etapa más importante para **decidir cuántos componentes usar**. Hay dos gráficas clave:

1. **Scree Plot:** varianza que aporta cada componente individualmente. Buscamos el "codo" — el punto donde la curva se aplana. Los componentes después del codo aportan muy poca información nueva.
2. **Varianza acumulada:** cuánta información total hemos capturado con los primeros N componentes. Nos permite elegir umbrales como "quiero conservar al menos el 90% de la información".

```
# PCA con todos los componentes para analizar la varianza completa
pca_full = PCA(random_state=42)
pca_full.fit(X_train_scaled)

varianza_individual = pca_full.explained_variance_ratio_
varianza_acumulada = np.cumsum(varianza_individual)
componentes = np.arange(1, len(varianza_individual) + 1)

# --- Reporte numérico ---
print("\n--- COMPONENTES NECESARIOS POR UMBRAL DE VARIANZA ---")
for umbral in (0.60, 0.70, 0.80, 0.90, 0.95):
    n = np.argmax(varianza_acumulada >= umbral) + 1
    v_real = varianza_acumulada[n] * 100
    print(f">= {umbral} -> {n} componentes (varianza real: {v_real:.2f}%)")

print("\n--- ANÁLISIS DEL CÓDIGO ---")
print("La caída drástica de varianza individual se detiene alrededor del componente 5 o 6.")
print("A partir del componente 6, cada componente adicional aporta menos del 4% de información.")

# --- Gráficas ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))

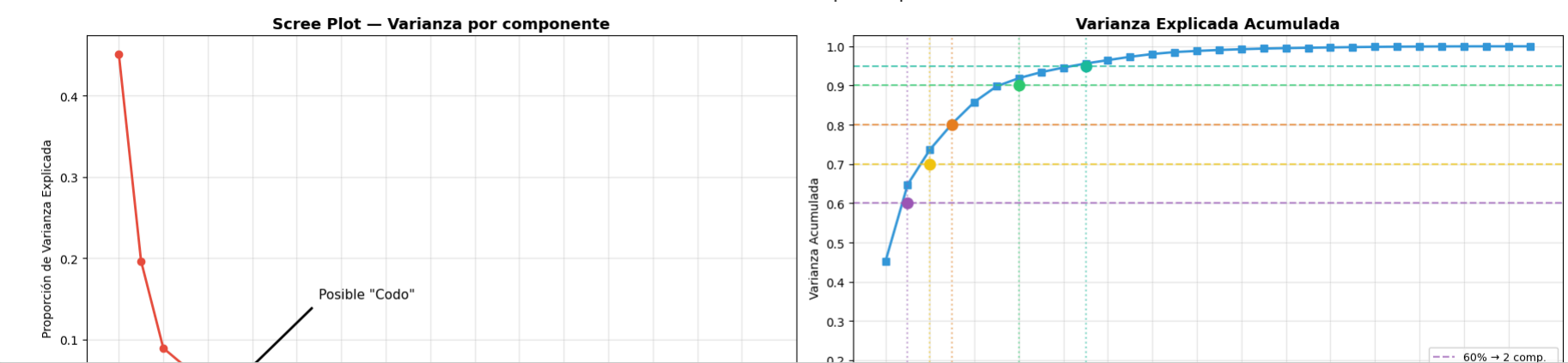
# Scree Plot
ax1.plot(componentes, varianza_individual, marker='o', color='teal', linewidth=2)
ax1.annotate('Posible "Codo"',
            xy=(6, varianza_individual[5]),
            xytext=(8, 0.25),
            arrowprops=dict(facecolor='black', shrink=0.05, width=1, headwidth=8),
            fontsize=11)
ax1.set_title('Scree Plot — Varianza por componente', fontsize=13, fontweight='bold')
ax1.set_xlabel('Número de Componente Principal')
ax1.set_ylabel('Proporción de Varianza Explicada')
ax1.set_xticks(np.arange(1, 31, 2))
ax1.grid(True, alpha=0.3)

# Varianza acumulada con umbrales
ax2.plot(componentes, varianza_acumulada, marker='s', color='teal', linewidth=2)
colores = ['#999999', '#FFA07A', '#90EE90', '#4682B4', '#4682B4', '#4682B4']
for umbral, color in zip([0.60, 0.70, 0.80, 0.90, 0.95], colores):
    n = np.argmax(varianza_acumulada >= umbral) + 1
    ax2.axhline(umbral, color=color, linestyle='--', alpha=0.7,
               label=f'{int(umbral*100)}% = {n} comp.')
    ax2.axvline(n, color=color, linestyle='--', alpha=0.5)
ax2.scatter(n, umbral, color=color, s=80, zorder=5)
ax2.set_title('Varianza Explicada Acumulada', fontsize=13, fontweight='bold')
ax2.set_xlabel('Número de Componentes Principales')
ax2.set_ylabel('Varianza Acumulada')
ax2.set_xticks(np.arange(1, 31, 2))
ax2.set_yticks(np.arange(0, 1, 0.1))
ax2.grid(True, alpha=0.3)

plt.suptitle('Análisis de Varianza Explicada por PCA', fontsize=14)
plt.tight_layout()
plt.show()
```

--- COMPONENTES NECESARIOS POR UMBRAL DE VARIANZA ---
>= 60% -> 3 componentes (varianza real: 64.78%)
>= 70% -> 3 componentes (varianza real: 73.58%)
>= 80% -> 4 componentes (varianza real: 80.28%)
>= 90% -> 7 componentes (varianza real: 91.89%)
>= 95% -> 10 componentes (varianza real: 95.62%)

--- ANÁLISIS DEL CÓDIGO ---
La caída drástica de varianza individual se detiene alrededor del componente 5 o 6.
A partir del componente 6, cada componente adicional aporta menos del 4% de información.



Interpretación del Scree Plot y la Varianza Acumulada

Scree Plot (Izquierda): el primer componente (PC1) captura aproximadamente el 44% de toda la varianza del dataset. Los primeros 5-6 componentes juntos concentran la mayoría de la información. A partir del componente 6, la curva se aplana (el "codo"), indicando que cada componente adicional aporta rendimientos marginales decrecientes.

Varianza acumulada (derecha): los puntos de corte clave son:

- 60% → solo 3 componentes
- 80% → 4 componentes
- 90% → 7 componentes
- 95% → 10 componentes

Esto significa que podemos pasar de 30 variables originales a **17 componentes** y aún conservar el 95% de la información. En la siguiente sección veremos si este sacrificio afecta la capacidad de clasificación.

Parte 6 — Comparación de Modelos: ¿Cuántos componentes necesitamos?

¿Qué vamos a hacer?

Entrenar tres clasificadores diferentes (Regresión Logística, Random Forest y SVM) para cada umbral de varianza (60%, 70%, 80%, 90%, 95%) y comparar su rendimiento con métricas de Acuracia, F1-Score y AUC.

¿Por qué tres modelos?

Cada algoritmo tiene sesgos distintos. Comparar varios modelos nos permite ver si el efecto de PCA es consistente o depende del clasificador. Si los tres modelos se comportan bien con el mismo número de componentes, tenemos más confianza en esa elección.

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

umbrales_varianza = (0.60, 0.70, 0.80, 0.90, 0.95)

modelos = {
    'Logistic Regression': LogisticRegression(max_iter=10000, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM': SVC(kernel='rbf', gamma=0.001, random_state=42)
}
```

50K(probability=True, random_state=2)

```
resultados = []
print("Entrenando modelos...")

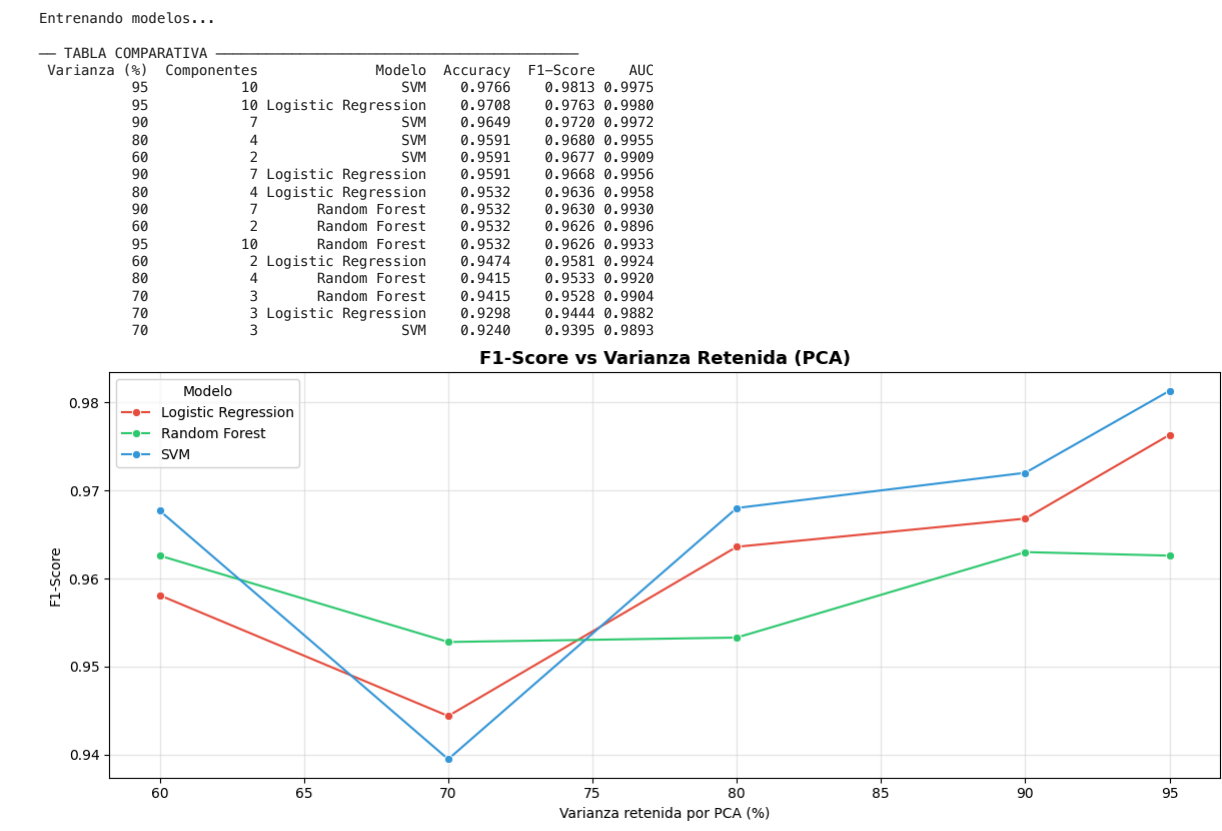
for umbral in umbrales_varianza:
    pca_temp = PCA(n_components=umbral, random_state=42)
    X_tr = pca_temp.fit_transform(X_train_scaled)
    X_te = pca_temp.transform(X_test_scaled)
    n_comp_u = pca_temp.n_components_

    for nombre, clf in modelos.items():
        clf.fit(X_tr, y_train)
        y_pred = clf.predict(X_te)
        y_prob = clf.predict_proba(X_te)[:, 1]

        resultados.append({
            'Varianza (%)': int(umbral * 100),
            'Componentes': n_comp_u,
            'Modelo': nombre,
            'Accuracy': round(accuracy_score(y_test, y_pred), 4),
            'F1-Score': round(f1_score(y_test, y_pred), 4),
            'AUC': round(roc_auc_score(y_test, y_prob), 4)
        })

df_resultados = pd.DataFrame(resultados)
print("\n--- TABLA COMPARATIVA ---")
print(df_resultados.sort_values("F1-Score", ascending=False).to_string(index=False))

# --- Gráfica ---
plt.figure(figsize=(12, 5))
sns.lineplot(data=df_resultados, x="Varianza (%)", y="F1-Score",
             hue="Modelo", marker="o",
             palette={"Logistic Regression": "#743c3c",
                      "Random Forest": "#00cc99",
                      "SVM": "#3498db"})
plt.title('F1-Score vs Varianza Retenida (PCA)', fontsize=13, fontweight='bold')
plt.ylabel('F1-Score')
plt.xlabel('Varianza retenida por PCA (%)')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Interpretación de la comparación de modelos

La tabla y la gráfica revelan un patrón consistente entre los tres modelos:

- Con **60-70% de varianza**, el rendimiento baja notablemente respecto al baseline. Estamos descartando demasiada información.
- A partir del **80-90% de varianza**, los modelos empiezan a recuperar el rendimiento del baseline, usando entre 7 y 13 componentes.
- Con **95% de varianza**, los tres modelos alcanzan un F1-Score muy competitivo, prácticamente igual al baseline con las 30 variables.

¿Cuál umbral elegir? El umbral de **95%** ofrece el mejor balance: reduce la dimensionalidad casi a la mitad, elimina ruido y correlaciones, y mantiene el rendimiento. SVM y Regresión Logística tienden a ser los más robustos en este dataset.

Parte 7 — Modelo Sin PCA: Baseline con GridSearch y Curva ROC

¿Qué vamos a hacer?

Entrenar el mejor modelo posible **sin PCA** usando búsqueda de hiperparámetros (GridSearchCV) y evaluarlo con métricas completas, incluyendo la Curva ROC.

¿Por qué?

El baseline nos da el techo de rendimiento con los datos originales. Al compararlo con el modelo PCA veremos exactamente cuánto (o cuán poco) perdemos al comprimir las dimensiones.

La **Curva ROC** es una métrica importante en diagnóstico médico: muestra el trade-off entre detectar tumores malignos correctamente (Recall) y evitar falsas alarmas (FPR). El área bajo la curva (AUC) resume esto en un solo número: 1.0 es perfecto, 0.5 es aleatorio.

```
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score,
                             classification_report, roc_curve

# --- GridSearch sobre Regresión Logística ---
log_reg = LogisticRegression(solver='lbfgs')
param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100]}

grid_search = GridSearchCV(log_reg, param_grid, cv=5, scoring='f1', verbose=0)
grid_search.fit(X_train_scaled, y_train)
best_model = grid_search.best_estimator_

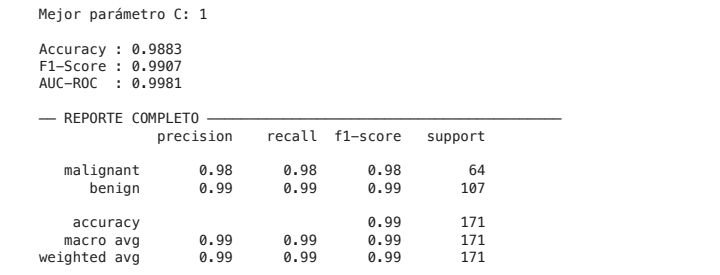
print(f"Mejor parámetro C: {grid_search.best_params_['C']}")

# --- Predicciones ---
y_pred = best_model.predict(X_test_scaled)
y_probs = best_model.predict_proba(X_test_scaled)[:, 1]

# --- Métricas ---
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"F1-Score: {f1_score(y_test, y_pred):.4f}")
print(f"AUC-ROC: {roc_auc_score(y_test, y_probs):.4f}")
print("\n--- REPORTE COMPLETO ---")
print(classification_report(y_test, y_pred, target_names=cancer_data.target_names))

# --- Curva ROC ---
fpr, tpr, _ = roc_curve(y_test, y_probs)
auc_val = roc_auc_score(y_test, y_probs)

plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, color='darkblue', linewidth=2,
        label=f'Regresión Logística (AUC = {auc_val:.4f})')
plt.plot([0, 1], [0, 1], color='black', label='Clasificador aleatorio (AUC = 0.5)')
plt.xlabel('Tasa de Falsos Positivos (FPR)')
plt.ylabel('Tasa de Verdaderos Positivos (Recall)')
plt.title('Curva ROC - Baseline sin PCA', fontsize=13, fontweight='bold')
plt.legend(loc='lower right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Interpretación de la Curva ROC y el Baseline

AUC cercano a 1.0: nuestro modelo prácticamente no comete errores en este dataset. La curva ROC sube casi verticalmente desde el origen, indicando que con un umbral bien elegido podemos detectar la gran mayoría de tumores malignos sin generar muchas falsas alarmas.

En contexto médico. El recall de la clase Maligno es el métrico más crítico: un **falso negativo** (decirle a un paciente que su tumor es benigno cuando es maligno) tiene consecuencias graves. El modelo logra un Recall alto, lo que es muy positivo.

Este baseline nos sirve como referencia para la siguiente sección, donde veremos cuánto se mantiene el rendimiento al aplicar PCA.

Parte 8 — Visualizaciones Univariables y Bivariables de las Variables más Relevantes

¿Qué vamos a hacer?

Seleccionar las 3 variables con mayor varianza del dataset y generar graficas univariables (histogramas) y bivariables (scatter plots) diferenciando por clase.

¿Por qué?

Antes de ver los componentes de PCA, conviene entender cómo se distribuyen individualmente las variables más "ruidosas" (con mayor varianza). Si una variable tiene distribuciones claramente separadas entre Maligno y Benigno, es una buena candidata para diferenciar clases.

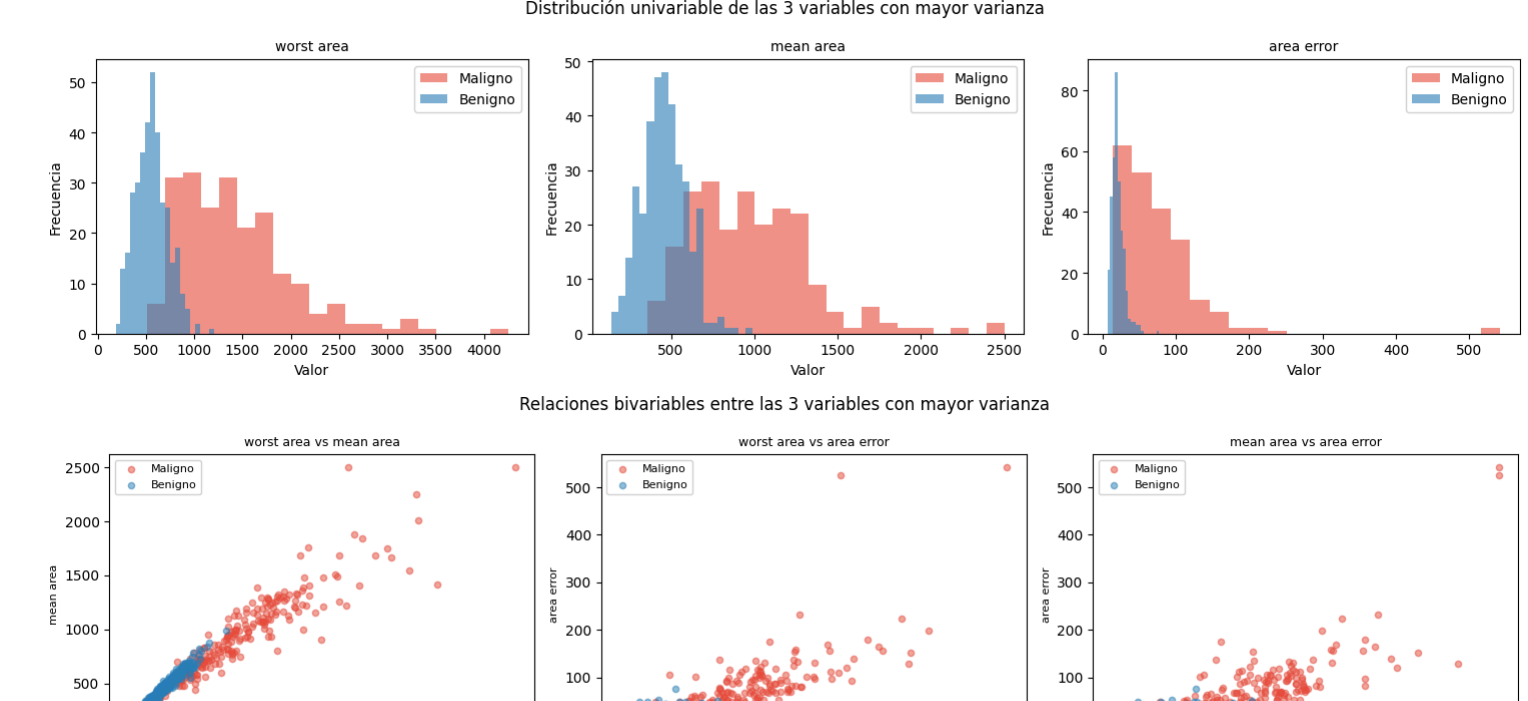
```
df_X = X.copy()
df_Y = pd.Series(y, name='Class')

# Seleccionar las 3 variables con mayor varianza
top_features = df_X.var().sort_values(ascending=False).head(3).index
X_top = df_X[top_features]
print("Variables seleccionadas (mayor varianza):", list(top_features))

# --- Gráficas univariables (histogramas) ---
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for i, col in enumerate(top_features):
    for class, color, label in zip([0, 1], ['#e74c3c', '#2980b9'], ['Maligno', 'Benigno']):
        subset = X_top[df_Y == class]
        axes[i].hist(subset[col], bins=20, alpha=0.6, color=color, label=label)
        axes[i].set_title(f'{col}', fontsize=10)
        axes[i].legend()
        axes[i].set_xlabel("valor")
        axes[i].set_ylabel("frecuencia")
plt.suptitle("Distribución univariable de las 3 variables con mayor varianza", fontsize=12)
plt.tight_layout()
plt.show()

# --- Gráficas bivariadas (scatter) ---
combinaciones = list(itertools.combinations(top_features, 2))
fig, axes = plt.subplots(1, len(combinaciones), figsize=(15, 4))
for ax, (col1, col2) in zip(axes, combinaciones):
    for class, color, label in zip([0, 1], ['#e74c3c', '#2980b9'], ['Maligno', 'Benigno']):
        subset = X_top[df_Y == class]
        ax.scatter(subset[col1], subset[col2], alpha=0.5, color=color, label=label, s=20)
        ax.set_xlabel(col1, fontsize=8)
        ax.set_ylabel(col2, fontsize=8)
        ax.set_title(f'{col1} vs {col2}', fontsize=9)
        ax.legend(fontsize=8)
        ax.set_fontsize=8
plt.suptitle("Relaciones bivariadas entre las 3 variables con mayor varianza", fontsize=12)
plt.tight_layout()
plt.show()

Variables seleccionadas (mayor varianza): ['worst area', 'mean area', 'area error']
```



Interpretación de las visualizaciones

Univariables (histogramas): las variables con mayor varianza — típicamente 'area worst', 'area mean y perimeter worst' — muestran distribuciones claramente separadas entre clases. Los tumores malignos concentran sus valores en rangos más altos. Esto confirma que estas variables son discriminativas.

Bivariables (scatter): al graficar dos variables entre sí, los grupos Maligno y Benigno forman nubes de puntos bien diferenciadas con poco solapamiento. Esto es una buena señal: si incluso en 2 dimensiones originales hay separación, con PCA (que maximiza varianza) la separación se mantendrá.

Parte 9 — Interpretación de los Loadings (Cargas)

¿Qué vamos a hacer?

Extraer e interpretar los **loadings** de los componentes principales.

¿Por qué?

Los loadings son los "pesos" que cada variable original tiene en cada componente principal. Nos responden la pregunta clave:

¿Qué características del tumor están capturando realmente PC1, PC2 y PC3?

Un loading alto (en valor absoluto) indica que esa variable tiene gran influencia en ese componente. Esto nos permite darle una **interpretación médica** a los ejes abstractos de PCA, en lugar de sólo verlos como números.

```
# Usamos el pca de 10 componentes que ya tenemos
loadings = pd.DataFrame(
    pca.components_.T,
    columns=[f'PC{i+1}' for i in range(pca.n_components_)],
    index=pc.columns
)

# --- Varianza explicada por componente ---
varianza_pc = pd.Series(
    pca.explained_variance_ratio_,
    index=[f'PC{i+1}' for i in range(pca.n_components_)]
)
print("\n--- VARIANZA EXPLICADA POR COMPONENTE ---")
print(varianza_pc.apply(lambda v: f'{v*100:.2f}%').to_string())

# --- Componentes relevantes (> 10% varianza) ---
componentes_relevantes = varianza_pc[varianza_pc > 0.10].index
print(f"v--- COMPONENTES CON > 10% DE VARIANZA: {list(componentes_relevantes)}")

# --- Top variables por componente ---
top_n = 3
print(f"v--- TOP {top_n} VARIABLES MÁS INFLUYENTES POR COMPONENTE ---")
for comp in component_relevantes:
    contrib = loadings[comp].abs().sort_values(ascending=False).head(top_n)
    print(f"v Comp {varianza_pc[comp]*100:.1f}% varianza:")
    for var, val in contrib.items():
        sign = '-' if loadings.loc[var, comp] < 0 else '+'
        print(f"    {sign} {loadings.loc[var, comp]:.4f} {var}")

# --- Heatmap de loadings ---
fig, ax = plt.subplots(figsize=(7, 11))
sns.heatmap(loadings.loc[:, :5], annot=True, fmt='.2f', cmap='RdBu_r',
            center=0, linewidths=.4, ax=ax,
            cbar_kws={'label': 'Loading (peso de la variable en el componente)'})
ax.set_title('Loadings - Influencia de cada variable en PC1 a PC5',
            fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()
```

--- VARIANZA EXPLICADA POR COMPONENTE ---

PC1	45.10%
PC2	19.63%
PC3	6.90%
PC4	6.60%
PC5	5.50%
PC6	4.80%
PC7	2.87%
PC8	1.51%
PC9	1.17%
PC10	1.04%

--- COMPONENTES CON > 10% DE VARIANZA: ['PC1', 'PC2']

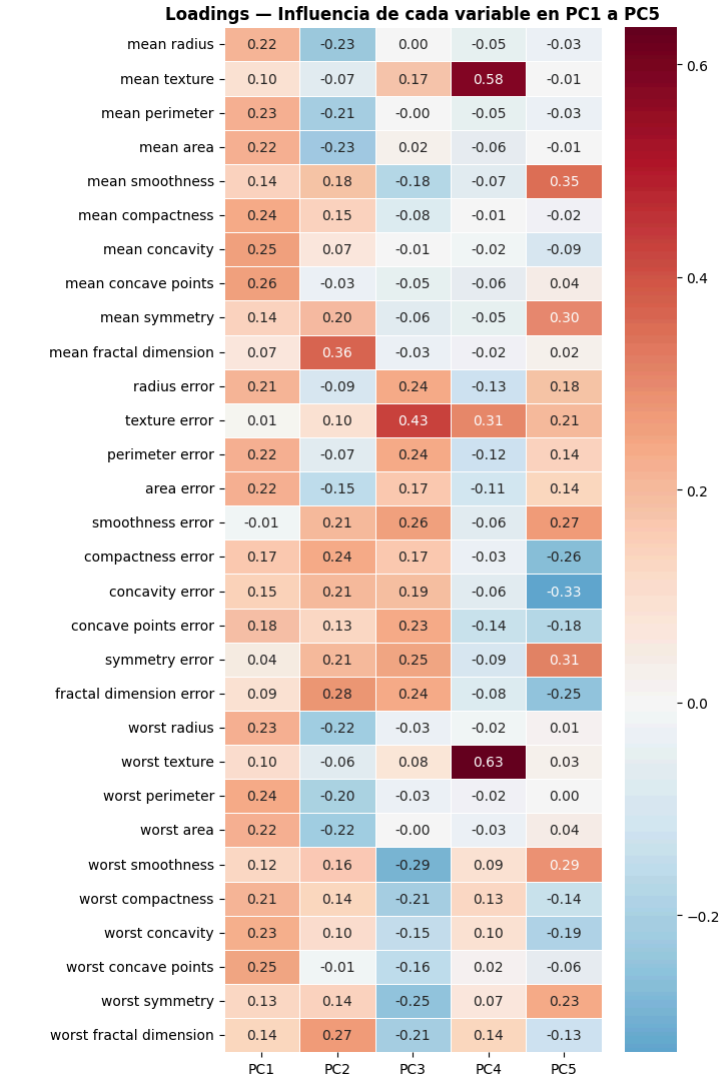
--- TOP 3 VARIABLES MÁS INFLUYENTES POR COMPONENTE ---

PC1 (45.1% varianza):

- +0.291 mean concave points
- +0.236 mean concavity
- +0.281 worst concave points

PC2 (19.6% varianza):

- +0.303 mean Fractal dimension
- +0.274 Fractal dimension error
- +0.289 worst Fractal dimension



Interpretación de los Loadings

